

# LISTA ENLAZADA CIRCULAR

---

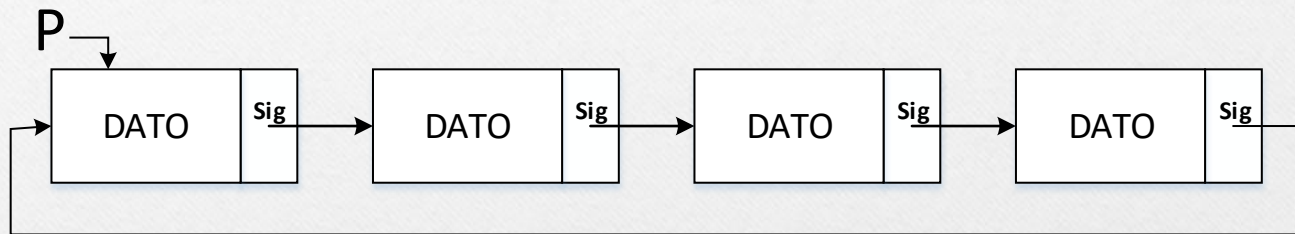
ESTRUCTURA DE DATOS Y ALGORITMOS

INF - 131

Lic. Marcelo Aruquipa

# LISTA SIMPLE CIRCULAR

- Lista que se caracteriza por que el ultimo nodo se enlaza con el primer nodo.



**Donde:**

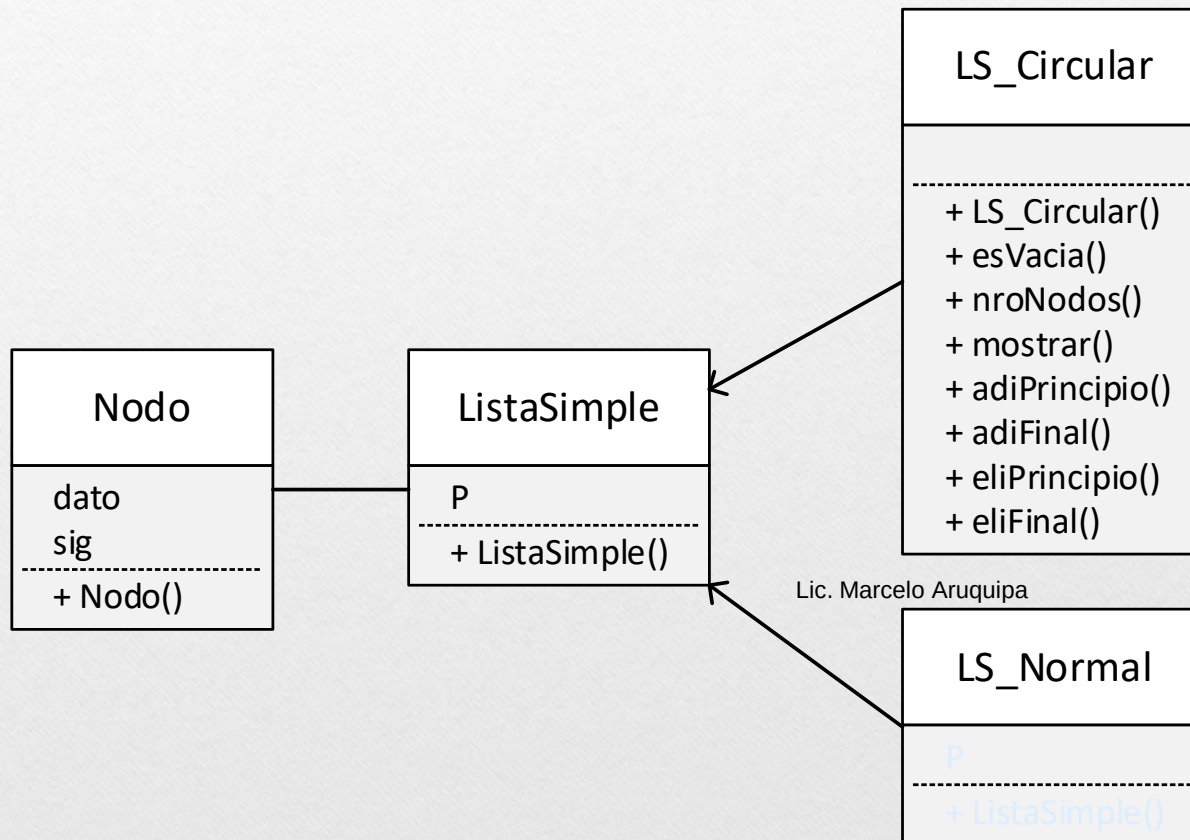
- **P** denominado Nodo Raíz.
- **DATO** contenido del Nodo
- **Sig** enlace al siguiente Nodo, el último Nodo apunta al primer Nodo

Lic. Marcelo Aruquipa



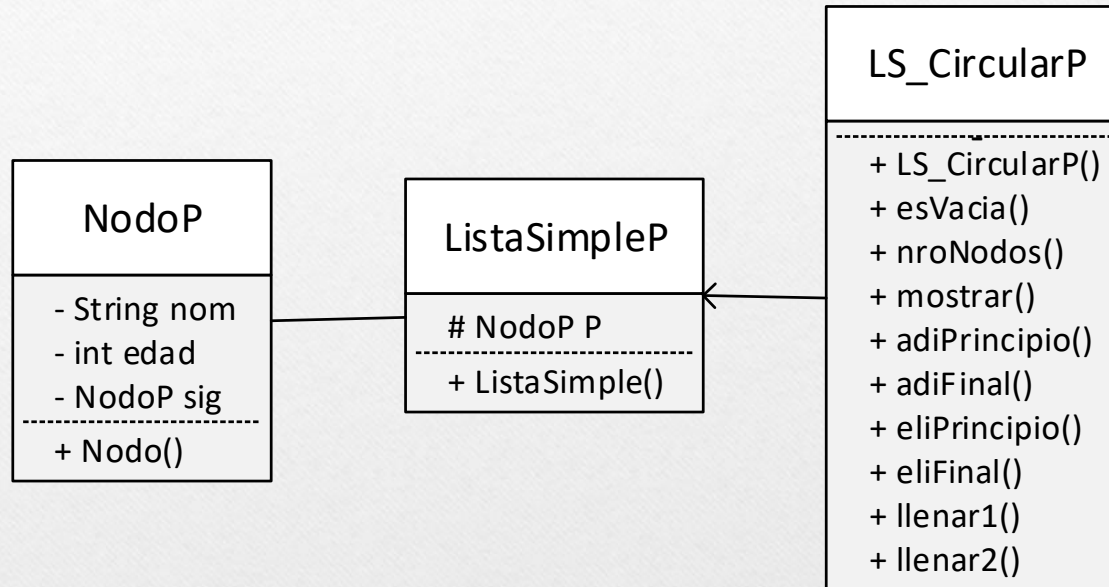
# LISTA SIMPLE CIRCULAR

## Diagrama de Clases



# LISTA SIMPLE CIRCULAR

**Ejemplo.** Implementar una lista de datos Persona.



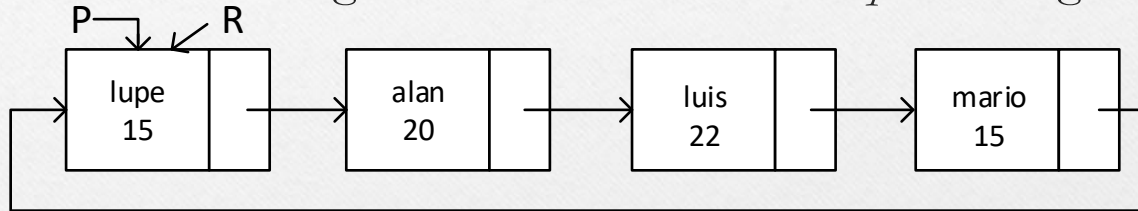
Lic. Marcelo Aruquipa

# LISTA SIMPLE CIRCULAR

Recorrer la lista

$R \leftarrow R.\text{sig}$

*“R tiene la dirección del siguiente **Nodo**”*   ó   *“R apunta al siguiente **Nodo**”*



Esto se puede realizar mientras el sig de **R** sea distinto de **P**

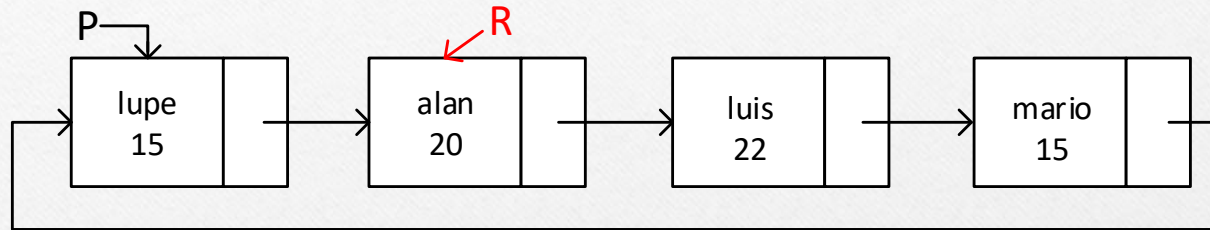
```
while(R.getSig() != P)
{
    R <- R.getSig();
}
```

Lic. Marcelo Aruquipa



# LISTA SIMPLE CIRCULAR

## Recorrer una lista



### Observaciones:

- ✓ Hacer notar que el avance en la lista es en una sola dirección (derecho), es decir que no se puede retroceder.
- ✓ Se puede utilizar mas de un puntero auxiliar.
- ✓ Si el puntero Raiz (**P**) se pierde se perderá toda la lista.
- ✓ El final de la lista sera cuando el sig del último nodo apunta al primer nodo.

Lic. Marcelo Aruquipa

# LISTA SIMPLE CIRCULAR

## Clase Nodo

```
class NodoP
{
    private:
        String nom;
        int edad;
        NodoP sig;
    public:
        NodoP()
        {
            sig <- null;
        }
}
```

## Clase ListaSimpleP

```
class ListaSimpleP
{
    protected:
        NodoP P;
    public:
        ListaSimpleP()
        {
            P <- null;
        }
}
```

En la clase **NodoP** se define los datos de persona

Lic. Marcelo Aruquipa



# LISTA SIMPLE CIRCULAR

## Clase LS\_CircularP

```
class LS_CircularP : ListaSimpleP
{
    public:
        bool esVacia()
        {
            if(P = null)
                return true;
            return false;
        }
        int nroNodos()
        {
            int c <- 0;
            if(P != null){
                NodoP R <- P;
                while(R.getSig() != P)
                {
                    c++;
                    R <- R.getSig();
                }
                c++;
            }
            return c;
        }
}
```

```
    adiFinal(string nom, int edad)
    {
        nuevo <- new NodoP();
        nuevo.setNom(nom);
        nuevo.setEdad(edad);
        if(P = null){
            P <- nuevo;
            P.setSig(P);
        }
        else{
            NodoP R <- P;
            while(R.getSig() != P)
            {
                R <- R.getSig();
            }
            R.setSig(nuevo);
            nuevo.setSig(P);
        }
    }
}
```

Lic. Marcel

```
    mostrar()
    {
        NodoP R <- P;
        while(R.getSig() != P)
        {
            print(R.getNom(), R.getEdad());
            R <- R.getSig();
        }
    }
    adiPrincipio(string nom, int edad)
    {
        nuevo <- new NodoP();
        nuevo.setNom(nom);
        nuevo.setEdad(edad);
        if(P = null){
            P <- nuevo;
            P.setSig(P);
        }
        else{
            NodoP R <- P;
            while(R.getSig() != P)
            {
                R <- R.getSig();
            }
            R.setSig(nuevo);
            nuevo.setSig(P);
            P <- nuevo;
        }
    }
}
```



# LISTA SIMPLE CIRCULAR

## Clase LS\_CircularP

```
NodoP eliPrincipio()
{
    x <- new NodoP();
    if(!esVacia())
    {
        if(P.getSig() = P)
        {
            x <- P;
            x.setSig(null);
            P <- null;
        }
        else{
            x <- P;
            P <- P.getSig();
            NodoP R <- P;
            while(R.getSig() != P)
            {
                R <- R.getSig();
            }
            R.setSig(P);
            x.setSig(null);
        }
    }
    return x;
}
```

```
NodoP eliFinal()
{
    ....
}
```

Ejercicio.

*Realiza la implementación  
Del método eliFinal()*

Lic. Marcelo Aruquipa

# LISTA SIMPLE CIRCULAR

## Clase LS\_CircularP

```
llenar1(int n)
{
    string nom;
    int edad;
    for (i <- 1 to n)
    {
        read(nom,edad);
        adiFinal(nom,edad);
    }
}

llenar2(int n)
{
    string nom;
    int edad;
    for (i <- 1 to n)
    {
        read(nom,edad);
        adiPrincipio(nom,edad);
    }
}

} //fin clase LS_CircularP
```

```
main()
{
    A <- new LS_CircularP();
    A.llenar1(5);
    A.mostrar();

    Persona z <- A.eliPrincipio();
    z.mostrar();

    A.adiFinal("Marcelo",25);
    A.mostrar();
}
```

Lic. Marcelo Aruquipa



# Actividad

---

Implementar el método eliFinal.

Implementar la lista de datos persona